

HARAS ADHAM

Excellence Équestre Marocaine

DOCUMENTATION TECHNIQUE COMPLÈTE

Site web — haras-adham-nypr.vercel.app

Version 3.0 — Juin 2026

Cette version documente la refonte complète du site : application Next.js (App Router) avec base de données Supabase, espace d'administration sécurisé, gestion de contenu multilingue et déploiement continu sur Vercel.

Elle remplace intégralement la version 2.0 qui décrivait l'ancien site mono-fichier HTML.

1. Vue d'ensemble du projet

1.1 Présentation générale

Haras Adham est le site officiel d'un haras marocain spécialisé dans l'élevage, la compétition et la préservation du Cheval Barbe. Le site est une application web moderne Next.js, rendue côté serveur, adossée à une base de données Supabase. Le contenu (chevaux, étalons, actualités, galerie, etc.) est entièrement géré depuis un espace d'administration protégé par mot de passe.

Propriété	Valeur
URL de production	https://haras-adham-nypr.vercel.app/
Dépôt Git	github.com/safiaala/haras-adham
Type	Application Next.js 16 (App Router), rendu serveur + composants client
Base de données	Supabase (PostgreSQL managé) + stockage de fichiers
Hébergement	Vercel (déploiement continu depuis Git, CDN mondial, HTTPS auto)
Langue principale	Français (FR)
Langues disponibles	FR / EN / ES / AR (avec support RTL pour l'arabe)
Email de contact	contact@harasadham.ma (configurable)
Horaires	Configurables depuis l'admin

1.2 Objectifs fonctionnels

- Présenter le haras et la race Barbe (histoire, classement UNESCO, Tbourida)
- Gérer un catalogue de chevaux avec filtres (sexe, discipline), recherche et fiche détaillée
- Gérer un catalogue d'étalons avec fiche dédiée et page par étalon
- Détailler les prestations du haras (sections et photos configurables)
- Publier actualités, événements et offres d'emploi
- Galerie photos avec filtrage par catégorie (catégories traduisibles) et lightbox
- Formulaire de contact avec sélection d'indicatif pays et anti-spam
- Édition complète du contenu et des photos via l'espace d'administration
- Site multilingue FR / EN / ES / AR commutable en temps réel
- Export des données du site au format CSV / Excel

2. Architecture technique

2.1 Stack technologique

Technologie	Rôle
Next.js 16 (App Router)	Framework React — routage, rendu serveur, routes API
React 19	Composants d'interface (client et serveur)
TypeScript	Typage statique de tout le code
Supabase (PostgreSQL)	Base de données, sécurité par lignes (RLS), stockage de fichiers
Cloudinary	Hébergement et optimisation des images (upload signé côté serveur)
Upstash Redis	Limitation de débit sur l'authentification et le contact
Resend	Envoi des emails du formulaire de contact

Technologie	Rôle
Vercel	Hébergement, déploiement continu, CDN, Analytics
Google Fonts	Noto Serif, Plus Jakarta Sans, Noto Sans Arabic, Material Symbols
OpenStreetMap	Carte de localisation du domaine (embed gratuit, sans clé API)

2.2 Structure des dossiers

Le code source est organisé selon les conventions du App Router de Next.js :

Chemin	Contenu
src/app/	Pages publiques et espace admin (un dossier = une route)
src/app/api/	Routes API serveur (authentification, données admin, contact...)
src/app/admin/	Pages de l'espace d'administration
src/components/	Composants réutilisables (Nav, Footer, listes, etc.)
src/lib/	Logique partagée : Supabase, traductions, authentification, types
supabase/	Scripts SQL (schéma, RLS, migrations)
middleware.ts	Protection des routes /admin (vérification du cookie de session)
next.config.ts	Configuration Next.js + en-têtes de sécurité (CSP, etc.)

2.3 Rendu et navigation

Contrairement à l'ancienne version (un seul fichier HTML), chaque page est désormais une route distincte servie par Next.js. La navigation utilise le routeur intégré (composant Link), avec préchargement automatique. Les données sont chargées depuis Supabase soit côté serveur, soit côté client selon les besoins de la page.

3. Base de données (Supabase)

3.1 Tables principales

Table	Contenu
chevaux	Catalogue des chevaux (nom, race, sexe, discipline, photos, prix...)
etalons	Catalogue des étalons reproducteurs
evenements	Agenda des événements
actualites	Articles d'actualité (avec champs traduits EN/ES/AR)
offres	Offres d'emploi
galerie	Photos du domaine (catégorie traduisible FR/EN/ES/AR)
config	Paires clé/valeur : coordonnées, réseaux sociaux, statistiques, hash du mot de passe admin
pages	Activation/désactivation et ordre des pages dans la navigation
sections	Blocs de contenu éditables (prestations, etc.)
messages	Messages du formulaire de contact (données personnelles, lecture serveur uniquement)

3.2 Sécurité au niveau des lignes (RLS)

La sécurité RLS (Row Level Security) est activée sur toutes les tables. Les règles appliquées :

- Lecture publique (anon) autorisée sur les tables de contenu affichées sur le site.
- Aucune écriture autorisée pour le rôle anonyme : toute création, modification ou suppression passe obligatoirement par les routes API serveur.
- Les écritures utilisent la clé `service_role` (secrète, côté serveur uniquement), après vérification de l'authentification administrateur.
- La table des messages de contact n'est lisible que côté serveur (`service_role`).

3.3 Stockage des images

Les images sont hébergées sur Cloudinary. L'upload se fait via une signature générée côté serveur (route `/api/admin/cloudinary-sign`) : le pré-réglage d'upload est en mode « Signed », ce qui empêche tout envoi non autorisé. Le bucket de stockage Supabase reste disponible en repli.

4. Pages publiques

Route	Fichier	Description
/	app/page.tsx	Accueil — hero, statistiques, aperçu chevaux, galerie...
/chevaux	app/chevaux/page.tsx	Catalogue des chevaux avec filtres et recherche
/etalons	app/etalons/page.tsx	Liste des étalons
/etalons/[id]	app/etalons/[id]/page.tsx	Fiche détaillée d'un étalon
/prestations	app/prestations/page.tsx	Présentation des prestations
/galerie	app/galerie/page.tsx	Galerie photos filtrable par catégorie
/actualites	app/actualites/page.tsx	Articles d'actualité
/evenements	app/evenements/page.tsx	Agenda des événements
/histoire	app/histoire/page.tsx	Histoire du haras et de la race Barbe
/jobs	app/jobs/page.tsx	Offres d'emploi
/contact	app/contact/page.tsx	Formulaire de contact + carte du domaine

5. Espace d'administration

5.1 Authentification

L'accès à `/admin` est protégé par mot de passe. Le mécanisme :

- La connexion se fait sur `/admin/login` via la route API `/api/admin/auth`.
- Le mot de passe n'est jamais stocké en clair : un jeton dérivé (SHA-256) est calculé et déposé dans un cookie sécurisé (`httpOnly`, `secure`, `sameSite=strict`, 7 jours).
- Le middleware protège toutes les pages `/admin` ; les routes API vérifient le cookie via la fonction `requireAdmin`.
- Une limitation de débit (Upstash) bloque les tentatives répétées : 5 essais par tranche de 10 minutes et par adresse IP.

5.2 Sections du tableau de bord

Section	Route	Rôle
Chevaux	/admin/chevaux	Gérer le catalogue des chevaux
Étalons	/admin/etalons	Gérer les étalons
Événements	/admin/evenements	Gérer l'agenda
Actualités	/admin/actualites	Gérer les articles (+ traductions)
Offres	/admin/offres	Gérer les offres d'emploi
Galerie	/admin/galerie	Gérer les photos et leurs catégories
Prestations	/admin/prestations	Gérer les sections de prestations
Pages	/admin/pages	Activer/désactiver pages + textes de la page d'accueil
Navigation	/admin/navigation	Ordonner le menu principal
Éditeur	/admin/editeur	Édition des blocs de contenu
Messages	/admin/messages	Consulter les messages du formulaire de contact
Config	/admin/config	Coordonnées, réseaux sociaux, GPS, mot de passe admin
Export CSV	/admin/export	Exporter les données en Excel / CSV

5.3 Changement de mot de passe

Le mot de passe administrateur peut être changé directement depuis l'interface, dans la page Config (section « Mot de passe administrateur »). Le nouveau mot de passe (8 caractères minimum) est dérivé en SHA-256 puis stocké dans la table config (clé `admin_password_hash`). Lors de la connexion, ce hash stocké est vérifié en priorité ; à défaut, la variable d'environnement `ADMIN_PASSWORD` sert de valeur initiale.

6. Routes API

Route	Méthodes	Rôle
/api/admin/auth	POST	Connexion admin (vérif. mot de passe + dépôt cookie)
/api/admin/logout	POST	Déconnexion (suppression du cookie)
/api/admin/change-password	POST	Changer le mot de passe administrateur
/api/admin/data	POST/PATCH/PUT/DELETE	CRUD générique sécurisé (tables autorisées)
/api/admin/messages	GET/PATCH/DELETE	Lecture et gestion des messages de contact
/api/admin/cloudinary-sign	POST	Signature serveur pour l'upload Cloudinary
/api/contact	POST	Réception du formulaire (+ email, anti-spam)

Toutes les routes `/api/admin` sont protégées par la vérification `requireAdmin`. La route `/api/admin/data` n'autorise les opérations que sur une liste blanche de tables.

7. Internationalisation (FR / EN / ES / AR)

Le site est disponible en quatre langues. Les textes d'interface sont centralisés dans `src/lib/translations.ts` (fonction `t(locale, clé)`). La langue est commutable en temps réel depuis la barre de navigation. L'arabe active la mise en page de droite à gauche (RTL).

- Textes d'interface : clés de traduction dans `translations.ts` (4 langues).
- Contenu de base de données traduisible : certains champs disposent de variantes `_en` / `_es` / `_ar` (ex. actualités, catégories de galerie).
- Noms de pays du formulaire de contact : traduits automatiquement via l'API navigateur `Intl.DisplayNames`.

8. Sécurité

- RLS activée sur toutes les tables ; aucune écriture anonyme possible.
- Mutations de données réservées au rôle `service_role`, côté serveur, après authentification.
- Cookie de session `httpOnly` + `secure` + `sameSite=strict` (protection CSRF).
- Déconnexion en POST uniquement (protection CSRF).
- Limitation de débit (Upstash) : 5 connexions / 10 min et 5 messages / heure par IP.
- Upload Cloudinary signé côté serveur (préréglage « Signed »).
- En-têtes de sécurité dans `next.config.ts` : CSP, X-Frame-Options, X-Content-Type-Options, Referrer-Policy, Permissions-Policy, HSTS.
- Clés secrètes (`service_role`, Cloudinary, Resend, Upstash) en variables d'environnement, jamais exposées au client.

9. Variables d'environnement

Variable	Usage
<code>NEXT_PUBLIC_SUPABASE_URL</code>	URL du projet Supabase (publique)
<code>NEXT_PUBLIC_SUPABASE_ANON_KEY</code>	Clé anonyme Supabase (lecture publique)
<code>SUPABASE_SERVICE_ROLE_KEY</code>	Clé <code>service_role</code> Supabase (secrète, écritures serveur)
<code>ADMIN_PASSWORD</code>	Mot de passe admin initial (repli si non changé via l'UI)
<code>NEXT_PUBLIC_CLOUDINARY_CLOUD_NAME</code>	Nom du cloud Cloudinary (public)
<code>NEXT_PUBLIC_CLOUDINARY_API_KEY</code>	Clé API Cloudinary (publique)
<code>CLOUDINARY_API_SECRET</code>	Secret Cloudinary (signature serveur)
<code>UPSTASH_REDIS_REST_URL</code>	URL Redis Upstash (rate limiting)
<code>UPSTASH_REDIS_REST_TOKEN</code>	Jeton Redis Upstash (secret)
<code>RESEND_API_KEY</code>	Clé API Resend (envoi des emails de contact)

10. Déploiement

Le site est déployé sur Vercel avec déploiement continu : chaque push sur la branche main du dépôt Git déclenche automatiquement un build et une mise en production.

- Commande de build : `next build --webpack` (le bundler webpack est forcé pour éviter un bug de Turbopack lié au fichier `middleware` lors du build de production).

- Les variables d'environnement sont configurées dans Vercel → Settings → Environment Variables.
- Après modification d'une variable d'environnement, un redéploiement est nécessaire.
- Les migrations SQL (dossier supabase/) doivent être exécutées manuellement dans Supabase → SQL Editor lors d'évolutions du schéma.

11. Sauvegarde et restauration

11.1 Sauvegarde des données

Le code source est versionné sur Git, mais le contenu réel du site (chevaux, étalons, actualités, galerie, configuration, messages) vit uniquement dans la base Supabase. Un script de sauvegarde exporte la totalité des tables :

- Commande : `node scripts/backup-supabase.mjs`
- Produit, dans `backups/<horodatage>/` : `data.json` (fidélité totale — tableaux et JSON conservés) et `restore.sql` (instructions INSERT réimportables).
- Le dossier `backups/` est volontairement exclu de Git (`.gitignore`) car la table `messages` contient des données personnelles et le dépôt est public.
- Les sauvegardes doivent être conservées hors du dépôt (disque externe ou cloud privé).

11.2 Restauration complète (recréation A→Z)

Pour reconstruire le site à l'identique à partir de zéro :

- 1. Récupérer le code (dossier du projet ou clone Git).
- 2. Renseigner les variables d'environnement (`.env.local` ou Vercel).
- 3. Créer le schéma : exécuter `supabase/schema.sql` puis `fix-rls-security.sql` dans Supabase.
- 4. Réinjecter les données : exécuter le fichier `restore.sql` de la sauvegarde.
- 5. Les images sont servies par Cloudinary via leurs URLs (compte à conserver).
- 6. Déployer (push Git → build Vercel automatique).

12. Maintenance courante

12.1 Modifier le contenu

Se connecter sur `/admin`, puis utiliser la section correspondante du tableau de bord. Aucune intervention sur le code n'est nécessaire pour le contenu courant.

12.2 Changer le mot de passe

Admin → Config → section « Mot de passe administrateur ».

12.3 Ajouter une migration de base de données

Créer un script `.sql` dans le dossier `supabase/`, puis l'exécuter dans l'éditeur SQL de Supabase. Exemple récent : `add-galerie-categorie-traductions.sql` (ajout des colonnes de traduction des catégories de galerie).